Container Technology

Docker & Kubernetes Administration & Security

Day 1

Container Fundamentals · Images & Builds · Container Security

Day 1 — Agenda

Part 1 — Container Basics and Administration

- Containers vs. VMs — use cases, tradeoffs, portability
- Container engines: Docker, containerd, CRI-O
- Management tools: crictl, nerdctl
- Linux kernel namespaces — the isolation primitives
- cgroups — resource control
- Union filesystems and image layers
- Images: pulling, inspecting, managing
- Running and limiting containers
- Troubleshooting

Part 2 — Images and Builds

- What is a container image?
- Dockerfile — instructions and best practices
- Multi-stage builds — keeping images small
- Build process deep dive
- Tagging strategies
- Container registries — DockerHub, GHCR, ECR, Harbor

Part 3 — Container Security

- Container threat model and attack surface
- SSL/TLS fundamentals
- TLS 1.3 handshake mechanics
- Mutual TLS (mTLS)
- Vulnerability scanning with Trivy
- CVE scoring — CVSS
- Image signing and verification — cosign and Sigstore
- Runtime security: seccomp, AppArmor, non-root, read-only fs

Part 1 — Container Basics and Administration

What is a Container?

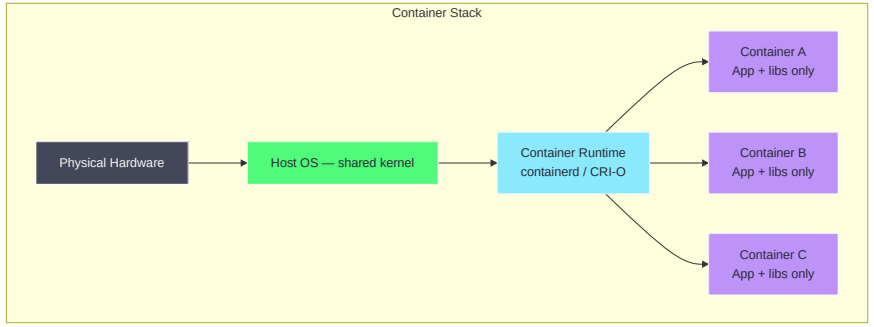
A container is a **lightweight, isolated process** running on a shared Linux kernel.

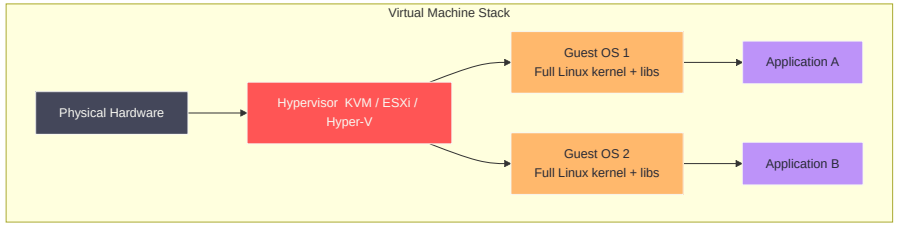
It is not a VM. It does not emulate hardware. It is a **process with constraints**.

Property	Containers	Virtual Machines
Kernel	Shared with host	Own kernel per VM
Boot time	Milliseconds	Seconds to minutes
Image size	MBs (often < 100 MB)	GBs
Isolation	Namespace + cgroups	Full hardware emulation
Portability	Build once, run anywhere	Hypervisor-specific
Overhead	Near-zero	10–30% CPU/RAM overhead

> Containers are not inherently less secure than VMs — but they do have a different and smaller attack surface when configured correctly.

Containers vs. VMs — Architecture





When to Use Containers vs. VMs

Use Case	Recommended
Microservices at scale	Containers
Legacy monoliths, untouched	VMs
Rapid CI/CD and GitOps	Containers
High-security multi-tenant isolation	VMs (or gVisor/Kata)
Immutable infrastructure	Containers
OS-level work (kernel modules, drivers)	VMs
Development environments (local)	Containers
Compliance requiring full OS isolation	VMs

> Many production environments use **both**: VMs for the cluster nodes, containers for the workloads running on them.

Container Engines — Overview

A **container engine** manages the full container lifecycle: pulling images, creating namespaces, applying cgroups, starting processes.

Engine	Maintained by	Used in
Docker Engine	Docker Inc.	Developer laptops, CI
containerd	CNCF	Kubernetes (default)
CRI-O	Red Hat / CNCF	OpenShift, K8s
Podman	Red Hat	RHEL, rootless workloads

> Kubernetes does **not** use the Docker Engine in production — it uses containerd or CRI-O via the **Container Runtime Interface (CRI)**. Docker on your laptop still builds images that run everywhere because images are an OCI standard.

containerd vs. CRI-O

Feature	containerd	CRI-O
Origin	Spun out of Docker	Red Hat for Kubernetes
CRI support	Via `cri` plugin	Native
Image support	OCI + Docker format	OCI only
Plugin ecosystem	Rich (snapshotter, etc.)	Minimal by design
Default in	EKS, GKE, AKS, upstream K8s	OpenShift, Fedora CoreOS
CLI (manual ops)	`nerdctl`, `ctr`	`crictl`

> Both are CNCF projects. The differences are operational, not functional — both run your containers correctly.

crictl and nerdctl

When the Docker CLI is not present on a node, you use:

crictl — talks directly to any CRI runtime

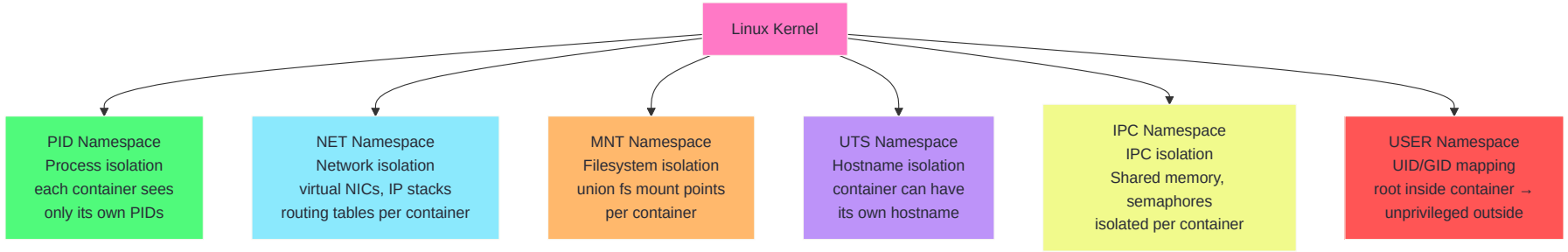
```
# List all pods and containers (like docker ps)
crictl pods
crictl ps

# Inspect a container
crictl inspect <container-id>

# Pull an image
crictl pull ubuntu:22.04

# Exec into a running container
crictl exec -it <container-id> /bin/bash
```

nerdctl — Docker-compatible CLI for containerd

```
nerdctl run -it --rm ubuntu:22.04 bash
nerdctl build -t myapp:latest .
nerdctl images
```

Linux Kernel Namespaces

Namespaces are the **core isolation mechanism** behind containers. Every container gets its own set of namespaces, making it believe it has dedicated resources.



Namespace Types — Deep Dive

Namespace	Isolates	`unshare` flag
PID	Process IDs — containers see PID 1	`--pid`
NET	Network interfaces, IP, routing, ports	`--net`
MNT	Filesystem mount points	`--mount`
UTS	Hostname and NIS domain name	`--uts`
IPC	SysV IPC, POSIX message queues	`--ipc`
USER	UID/GID mapping (root inside ≠ root outside)	`--user`
Cgroup	cgroup root — hides host cgroup tree	`--cgroup`
Time	Boot and monotonic clocks (Linux 5.6+)	`--time`

```
# Inspect namespaces of a running container
lsns -p $(docker inspect --format '{{.State.Pid}}' mycontainer)
```

Inspecting Namespaces in Practice

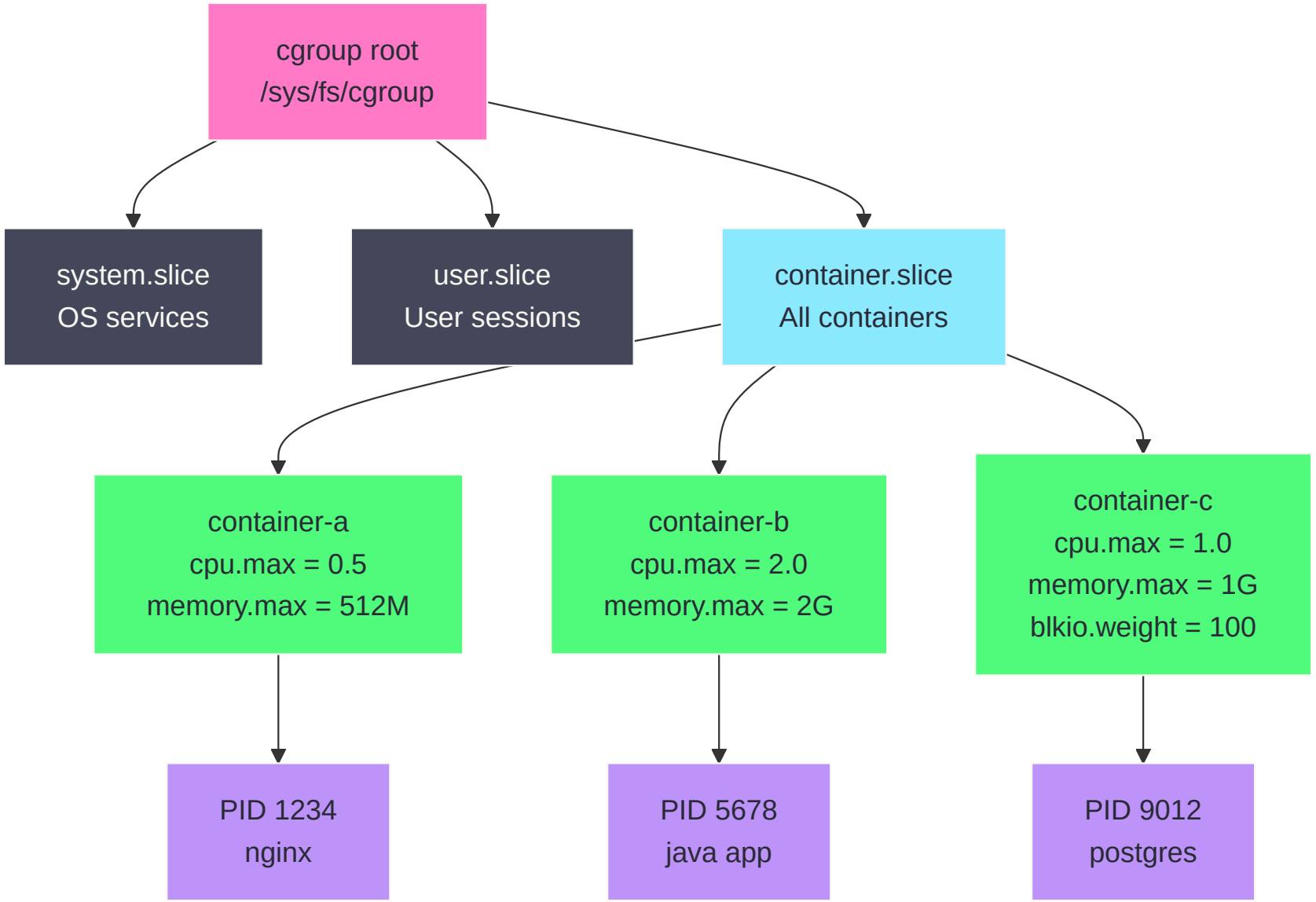
```
# Find the PID of a running container's init process
CPID=$(docker inspect --format '{{.State.Pid}}' my-nginx)

# List the namespace inodes
ls -la /proc/$CPID/ns/

# Compare: host PID namespace vs container PID namespace
ls -li /proc/1/ns/pid       # host
ls -li /proc/$CPID/ns/pid   # container (different inode = different namespace)

# Enter a container's network namespace (like nsenter)
nsenter --target $CPID --net ip addr
```

> Namespace inode numbers are how the kernel tracks which namespace a process belongs to. Two processes share a namespace if and only if their `/proc/PID/ns/X` symlinks point to the same inode.

cgroups — Resource Control

Control groups (cgroups) limit the resources a container can consume: CPU, memory, disk I/O, network.

cgroups v2 is the current standard (Linux 4.5+, default on all modern distros).


cgroup root
/sys/fs/cgroup
system.slice
OS services
user.slice
User sessions
container.slice
All containers
container-a
cpu.max = 0.5
memory.max = 512M
container-b
cpu.max = 2.0
memory.max = 2G
container-c
cpu.max = 1.0
memory.max = 1G
blkio.weight = 100
PID 1234
nginx
PID 5678
java app
PID 9012
postgres

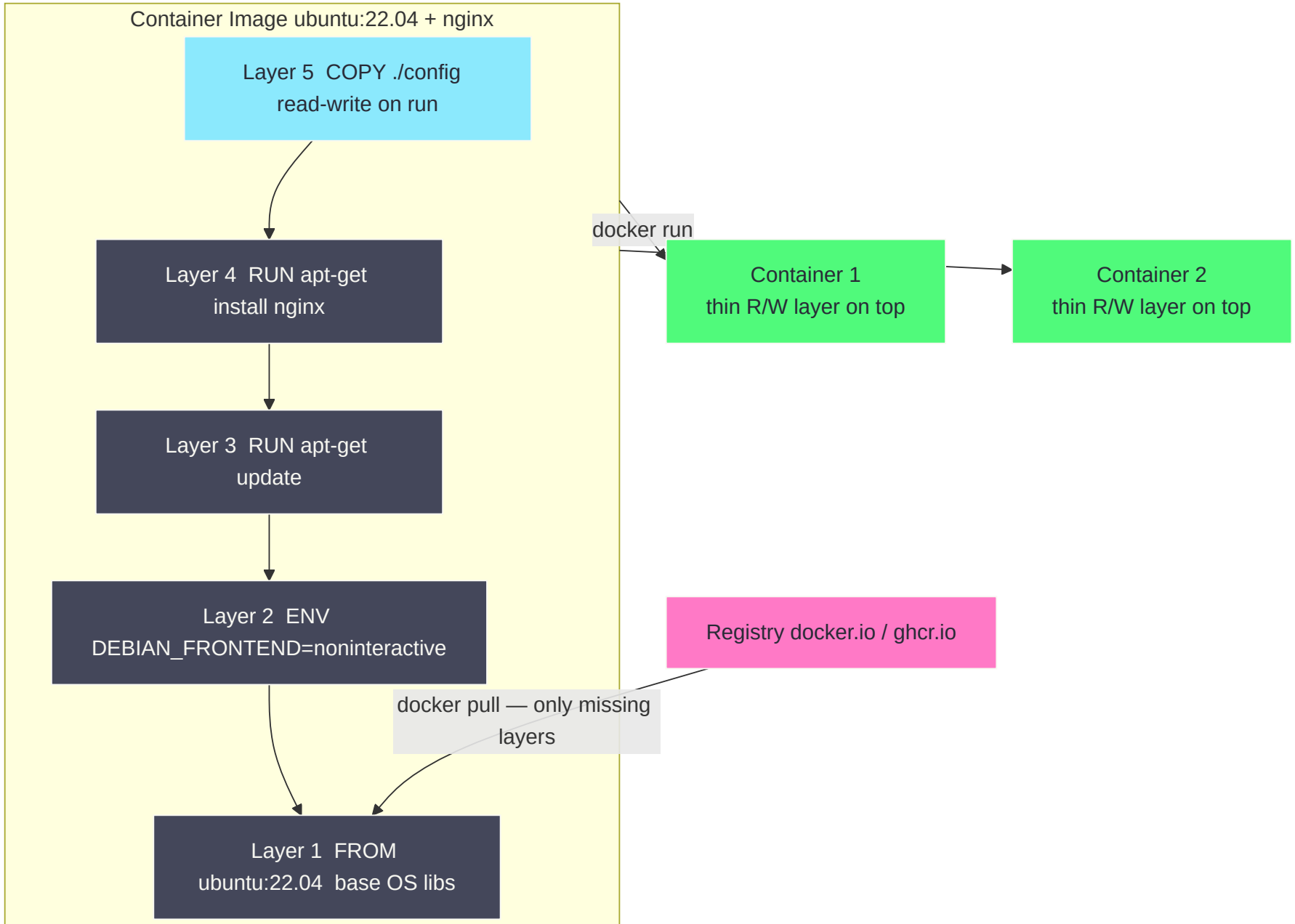
cgroups in Practice

```
# Docker flags map to cgroup settings
docker run \
  --memory="512m" \           # memory.max
  --memory-swap="512m" \      # no swap
  --cpus="1.5" \              # cpu.max = 150000 100000
  --pids-limit=100 \          # pids.max
  nginx

# Read cgroup settings for a running container
CPID=$(docker inspect --format '{{.State.Pid}}' nginx)
cat /proc/$CPID/cgroup

# Direct cgroup v2 file
cat /sys/fs/cgroup/$(cat /proc/$CPID/cgroup | cut -d: -f3)/memory.max
```

> Memory limits are enforced by the OOM killer. When a container exceeds `memory.max`, the kernel kills the largest process in the cgroup. You will see `OOMKilled: true` in `docker inspect`.

Union Filesystems and Image Layers

Container images are built from **read-only layers** stacked using a union filesystem (overlay2 on modern Linux).


Container Image ubuntu:22.04 + nginx
Layer 5 COPY ./config
read-write on run
Layer 4 RUN apt-get
install nginx
Layer 3 RUN apt-get
update
Layer 2 ENV
DEBIAN_FRONTEND=noninteractive
Layer 1 FROM
ubuntu:22.04 base OS libs
docker run
Container 1
thin R/W layer on top
Container 2
thin R/W layer on top
Registry docker.io / ghcr.io
docker pull — only missing
layers

How OverlayFS Works

```
lowerdir = image layers (read-only, shared between containers)
upperdir = container's writable layer
merged   = the union view the container sees
```

```
# Docker stores overlay mounts here
ls /var/lib/docker/overlay2/

# Inspect a container's graph driver details
docker inspect --format '{{json .GraphDriver}}' my-nginx | jq

# Shows: LowerDir, UpperDir, WorkDir, MergedDir
```

Key insight: When Container A and Container B both run from `ubuntu:22.04`, the base layers are on disk **once**, not twice. Storage efficiency scales with the number of containers sharing a base.

> Copy-on-Write (CoW): a file in `lowerdir` is only copied to `upperdir` when a container **writes** to it.

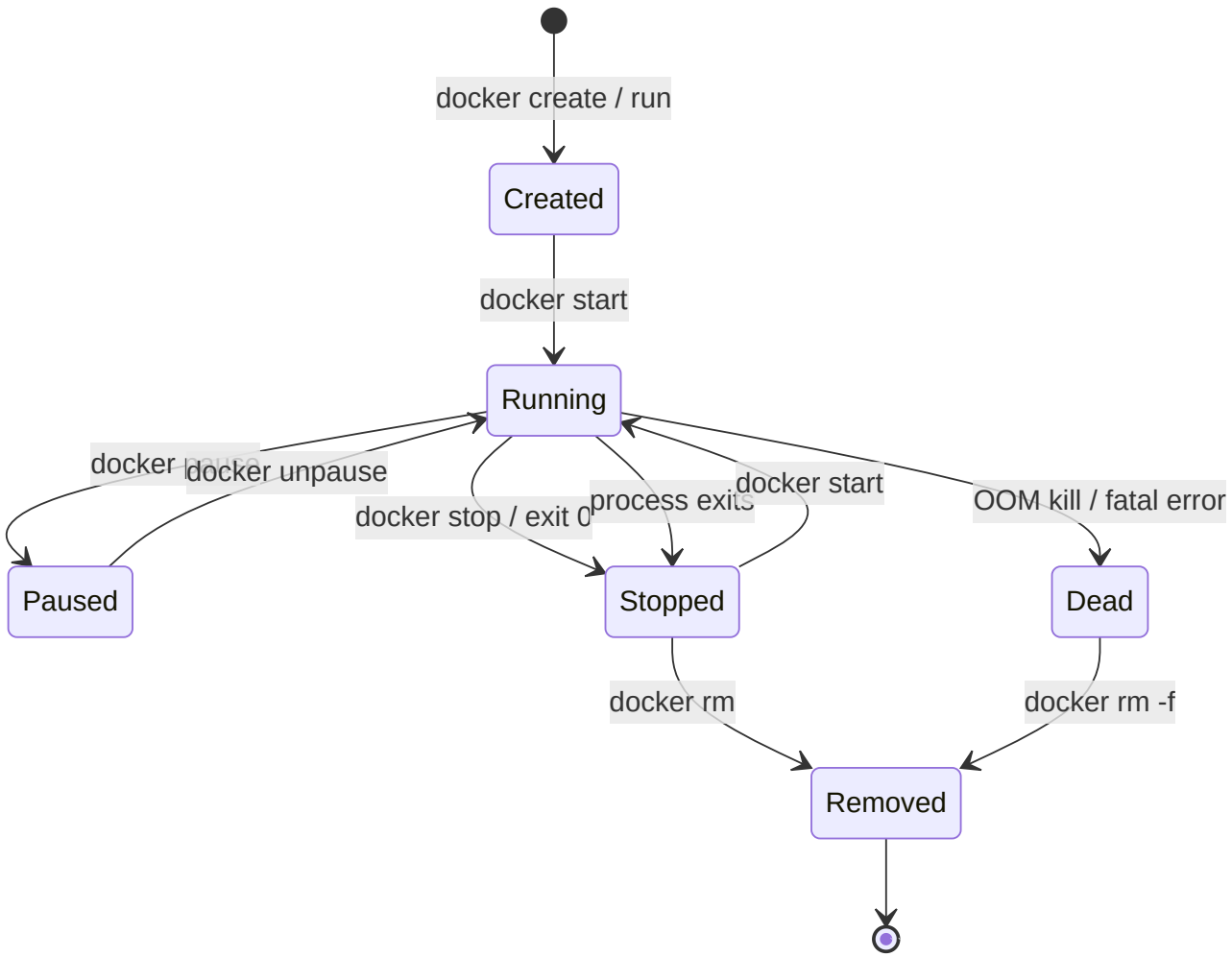
Images — Pulling, Inspecting, Managing

```
# Pull from Docker Hub (default) or a specific registry
docker pull ubuntu:22.04
docker pull ghcr.io/myorg/myapp:v1.2.3

# List images
docker images

# Inspect image metadata, layers, entrypoint
docker inspect ubuntu:22.04 | jq '.[0].Config'

# Show image history / layers
docker history ubuntu:22.04 --no-trunc

# Show disk usage
docker system df -v

# Remove dangling images (untagged)
docker image prune

# Remove all unused images
docker image prune -a
```

Running Containers

```docker
# Basic run (foreground)
docker run ubuntu:22.04 echo "hello"

# Detached with a name, port mapping, env var
docker run -d \
  --name webserver \
  -p 8080:80 \
  -e APP_ENV=production \
  nginx:1.27-alpine

# Interactive shell
docker run -it --rm ubuntu:22.04 bash

# Mount a volume
docker run -v /data/nginx:/etc/nginx/conf.d:ro nginx

# Override entrypoint
docker run --entrypoint /bin/sh nginx -c "nginx -t"
```

Container Lifecycle



Resource Limiting — Security Context

Always set resource limits. Unlimited containers are an availability risk.

```
docker run \
  --memory="256m" \
  --memory-swap="256m" \
  --cpus="0.5" \
  --pids-limit=50 \
  --read-only \
  --tmpfs /tmp:size=64m \
  --cap-drop ALL \
  --cap-add NET_BIND_SERVICE \
  --security-opt no-new-privileges:true \
  --user 1000:1000 \
  myapp:latest
```

> `--cap-drop ALL` removes all Linux capabilities. Re-add only what the app genuinely needs. Most apps need zero capabilities.

Troubleshooting Containers

```
# Container logs (last 100 lines, follow)
docker logs --tail=100 -f mycontainer

# Live resource usage
docker stats

# Inspect full container state
docker inspect mycontainer | jq '.[0].State'

# Exec into a running container
docker exec -it mycontainer /bin/sh

# Copy files out of a container (e.g., for forensics)
docker cp mycontainer:/var/log/app.log ./

# Events in real-time
docker events --filter container=mycontainer

# Inspect network
docker network inspect bridge

# Check why a container exited
docker inspect --format '{{.State.ExitCode}} {{.State.Error}}' mycontainer
```

Part 2 — Images and Builds

What is a Container Image?

A container image is an **immutable, layered archive** conforming to the OCI Image Specification.

It contains:

- A **manifest** — lists layers and their digests
- One or more **layer tarballs** — filesystem deltas
- A **config JSON** — entrypoint, env, exposed ports, user

```
# Every image has a content-addressable digest
docker inspect --format '{{.Id}}' ubuntu:22.04
# sha256:a6d6b4b47... — digest of the config JSON

# Pull by digest (immutable reference — good for production!)
docker pull ubuntu@sha256:a6d6b4b47...
```

> A tag like `ubuntu:22.04` is **mutable** — it can be overwritten. A digest is **immutable**. Pin digests in production manifests.

Dockerfile — Core Instructions

```dockerfile
# Base image — always pin a digest or specific tag
FROM ubuntu:22.04

# Metadata
LABEL maintainer="ops@example.com" version="1.0"

# Environment variables
ENV APP_HOME=/opt/app \
    PORT=8080

# Run commands — each RUN creates a layer
RUN apt-get update && apt-get install -y \
    curl \
    ca-certificates \
 && rm -rf /var/lib/apt/lists/*

# Copy files from build context
COPY --chown=app:app ./src $APP_HOME

# Working directory
WORKDIR $APP_HOME

# Non-root user
USER app

# Expose documentation (does not actually publish port)
EXPOSE 8080

# Default command
CMD ["./myapp"]
```

Dockerfile Best Practices

Practice	Why
Pin base image tags (or digests)	Reproducible builds
Combine `apt-get update && install && clean` in one `RUN`	Smaller layers, no stale cache
Use `.dockerignore`	Don't leak secrets or node_modules
One process per container	Simpler lifecycle, logging, scaling
`COPY` before installing deps	Better layer cache reuse
Run as non-root	Reduces blast radius if compromised
Use `--no-cache-dir` for pip	Smaller image
`HEALTHCHECK` instruction	Container orchestrators can restart unhealthy containers

```dockerfile
# Bad — two RUN commands = two layers, update cache stale in second
RUN apt-get update
RUN apt-get install -y nginx

# Good — single layer, clean apt cache
RUN apt-get update && apt-get install -y nginx && rm -rf /var/lib/apt/lists/*
```

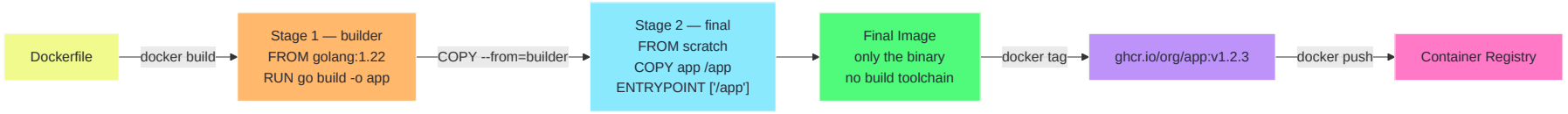
Multi-stage Builds

Multi-stage builds separate **build toolchain** from **runtime image**, producing the smallest possible final image.

```
# Stage 1 — builder (large, has Go compiler)
FROM golang:1.22-bookworm AS builder
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -trimpath -o /app ./cmd/server

# Stage 2 — final (tiny, no compiler)
FROM gcr.io/distroless/static-debian12
COPY --from=builder /app /app
USER nonroot:nonroot
ENTRYPOINT ["/app"]
```

> `distroless` images contain only your app and its runtime dependencies — no

Build Process Diagram



Build Arguments and Secrets

```
# Build-time variables (NOT secrets)
ARG APP_VERSION=dev
LABEL version="${APP_VERSION}"

# ✅ Mount secrets safely — NOT baked into layers
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc \
    npm install

# ✅ SSH agent forwarding for private repos
RUN --mount=type=ssh \
    git clone git@github.com:myorg/private-lib.git
```

```
docker build \
  --secret id=npmrc,src=$HOME/.npmrc \
  --ssh default \
  --build-arg APP_VERSION=1.2.3 \
  -t myapp:1.2.3 .
```

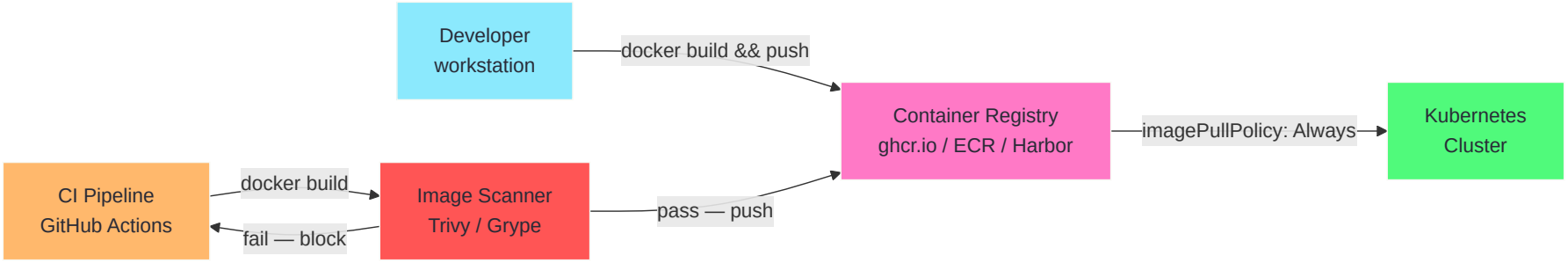
> Never use `ENV` or `ARG` to pass passwords. They appear in `docker history` and the image config — even if set in a layer that was later removed.

Tagging Strategies

```
# Semantic versioning — recommended
docker tag myapp:latest myapp:1.2.3
docker tag myapp:latest myapp:1.2
docker tag myapp:latest myapp:1

# Registry-qualified tags
docker tag myapp:1.2.3 ghcr.io/myorg/myapp:1.2.3

# Git SHA tag (for traceability)
docker tag myapp:latest myapp:$(git rev-parse --short HEAD)

# Push all tags
docker push ghcr.io/myorg/myapp:1.2.3
docker push ghcr.io/myorg/myapp:latest   # only on main branch!
```

Tag	Use
`:latest`	Local dev only — never deploy with this
`:1.2.3`	Production deployments
`:1.2.3-abc1234`	CI artifacts — version + git SHA
`@sha256:…`	Ultimate pinning — immutable

Container Registries



Registry Options

Registry	Hosted By	Authentication	Features
Docker Hub	Docker Inc.	PAT / OIDC	Rate limits on free tier
GHCR	GitHub	GitHub token	Native GitHub Actions integration
ECR	AWS	IAM roles	Lifecycle policies, image scanning
GCR / Artifact Registry	Google	Workload Identity	Binary Authorization
Harbor	CNCF / self-hosted	OIDC, LDAP	Scanning, replication, signing

```
# Log in to a registry
docker login ghcr.io -u USERNAME --password-stdin

# Use credential helpers (more secure than storing in ~/.docker/config.json)
# docker-credential-helper-ecr, docker-credential-gcr, etc.
```

> **Credential security:** `docker login` stores base64-encoded credentials in `~/.docker/config.json`. On production systems, use a credential helper or short-lived tokens.

Part 3 — Container Security

The Container Threat Model

Security is not a binary. Model the threats before choosing controls.

Threat	Example	Mitigation
Vulnerable base image	CVE in libc	Scan images, update regularly
Leaked secrets in image	API key in Dockerfile	Build secrets, secret management
Privileged container escape	`--privileged` misuse	Drop all caps, no privileged
Supply chain attack	Malicious dependency	Sign images, verify signatures
Resource exhaustion	Crypto miner	cgroup limits, PID limits

SSL/TLS — Why It Matters for Containers

Containers communicate across networks. Any unencrypted channel is a liability.

TLS protects:

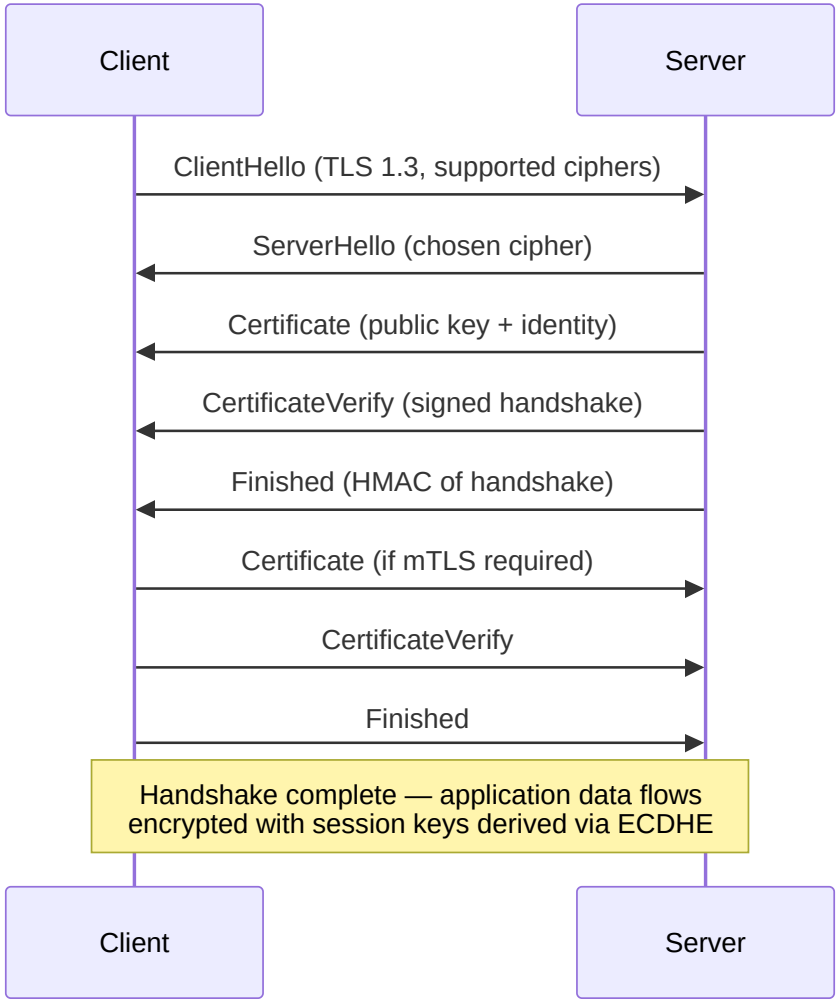
- Container registry authentication and image pulls
- API server communications in Kubernetes
- Service-to-service traffic (enforce via mTLS)
- Ingress traffic from users

TLS provides:

1. **Confidentiality** — encrypted payload
2. **Integrity** — MAC ensures data isn't tampered with
3. **Authentication** — certificate proves server identity

> Default Kubernetes cluster communications use TLS. Your applications must too.

TLS 1.3 Handshake



TLS 1.3 vs 1.2 — Key Improvements

Feature	TLS 1.2	TLS 1.3
Round trips to establish	2-RTT	1-RTT (0-RTT resumption)
Forward secrecy	Optional	Mandatory (ECDHE always)
Cipher suites	37 options (many weak)	5 secure suites only
RSA key exchange	Allowed	Removed
Vulnerable ciphers (RC4, 3DES)	Possible	Impossible

> Always disable TLS 1.0 and 1.1. Prefer TLS 1.3. For mTLS between services, use a service mesh (Istio, Linkerd) to automate certificate rotation.

Mutual TLS (mTLS)

Standard TLS: the **client verifies the server**.

mTLS: **both sides present and verify certificates** — no service can communicate unless it has a valid identity.

```
Container A ─── presents cert A ──▶ Container B
Container A ◀── presents cert B ─── Container B
          (both verified against same CA)
```

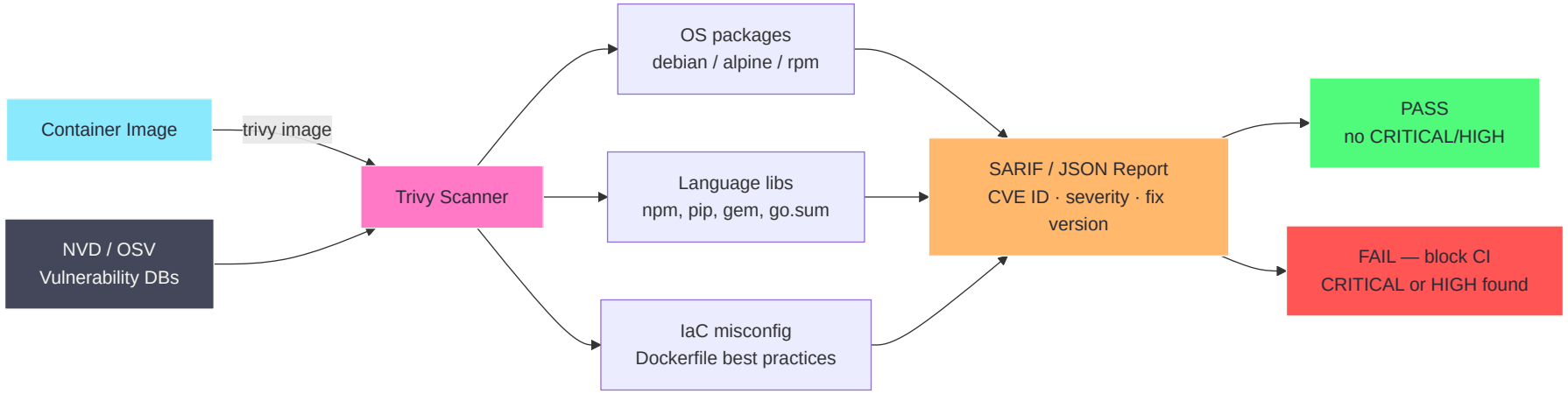
Why it matters for containers:

- Proves a pod's identity, not just its IP (IPs are ephemeral in K8s)
- Encrypts east-west (pod-to-pod) traffic automatically
- Eliminates need for app-level authentication between internal services

> In Kubernetes, a **service mesh** (Istio, Linkerd, Cilium) injects a sidecar proxy that handles mTLS transparently — no application code changes needed.

Vulnerability Scanning

Every container image is a software supply chain. Scan it.



Trivy in Practice

```
# Scan a local image
trivy image myapp:latest

# Scan with severity filter (fail CI on CRITICAL or HIGH)
trivy image --exit-code 1 --severity CRITICAL,HIGH myapp:latest

# Scan a Dockerfile for misconfigurations
trivy config ./Dockerfile

# Scan a filesystem (e.g., in CI before build)
trivy fs --security-checks vuln,secret .

# Output as SARIF for GitHub Security tab
trivy image --format sarif --output results.sarif myapp:latest

# Scan a remote image without pulling
trivy image --remote ghcr.io/myorg/myapp:latest
```

Understanding CVE Scores (CVSS v3)

Score Range	Severity	Action
9.0 – 10.0	**Critical**	Fix immediately, block deploy
7.0 – 8.9	**High**	Fix within 7 days
4.0 – 6.9	**Medium**	Fix within 30 days
0.1 – 3.9	**Low**	Schedule next release
0.0	None	Informational only

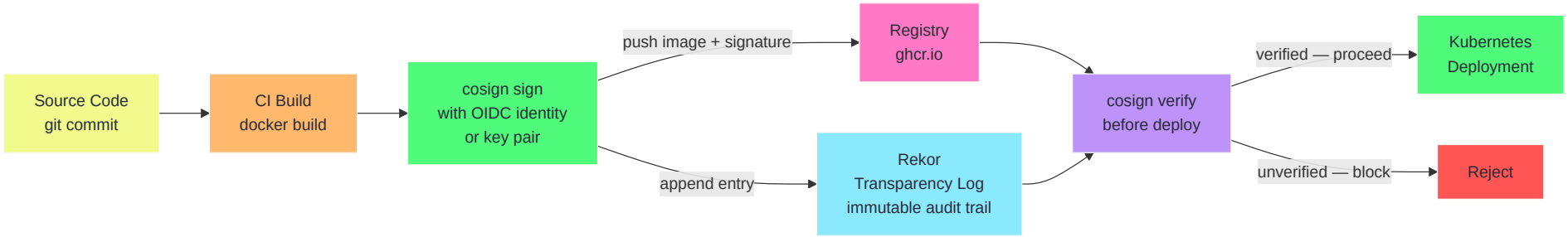
CVSS v3 components:

- **Attack Vector** — Network / Adjacent / Local / Physical
- **Attack Complexity** — Low / High
- **Privileges Required** — None / Low / High
- **User Interaction** — None / Required
- **Scope** — Unchanged / Changed
- **CIA Impact** — None / Low / High

> Not all CVEs are exploitable in your environment. Assess whether the vulnerable code path is actually reachable.

Image Signing and Verification

Signing proves an image was built by a trusted party and has not been tampered with.



cosign in Practice

```
# Generate a key pair (alternative: keyless OIDC signing)
cosign generate-key-pair

# Sign an image (key-based)
cosign sign --key cosign.key ghcr.io/myorg/myapp:1.2.3

# Sign keylessly (OIDC — in CI, no key to manage)
cosign sign ghcr.io/myorg/myapp:1.2.3
# Uses GitHub Actions / Google / Microsoft OIDC identity

# Verify before deploying
cosign verify \
  --certificate-identity-regexp "https://github.com/myorg/.*" \
  --certificate-oidc-issuer "https://token.actions.githubusercontent.com" \
  ghcr.io/myorg/myapp:1.2.3

# Enforce in Kubernetes via Policy Controller (Sigstore)
# All pods must have a valid cosign signature to be admitted
```

Runtime Security

Even with a good image, a container can be compromised at runtime. Apply defence-in-depth.

```
# 1. Drop ALL Linux capabilities, add back only what's needed
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE myapp

# 2. Run as non-root user
docker run --user 1001:1001 myapp

# 3. Read-only root filesystem
docker run --read-only --tmpfs /tmp myapp

# 4. No privilege escalation
docker run --security-opt no-new-privileges:true myapp

# 5. Apply a seccomp profile (block ~300 syscalls)
docker run --security-opt seccomp=/path/to/profile.json myapp

# 6. AppArmor profile
```

Linux Capabilities — What to Keep

Capabilities divide root's powers into granular units.

Capability	Needed for	Keep?
`NET_BIND_SERVICE`	Bind port < 1024	Only if needed
`CHOWN`	Change file ownership	Rarely
`DAC_OVERRIDE`	Bypass file permissions	Almost never
`NET_RAW`	Raw sockets (ping, packet capture)	No — drop always
`SYS_PTRACE`	Debug processes	No — serious escape risk
`SYS_ADMIN`	Huge — mount, cgroups, etc.	Never

```
# List capabilities of a running container
docker inspect --format '{{.HostConfig.CapAdd}} {{.HostConfig.CapDrop}}' mycontainer

# The default Docker capability set is already reduced from full root
# But --cap-drop ALL is the only safe starting point
```

seccomp Profiles

Seccomp (Secure Computing Mode) filters which Linux system calls a container can make.

```
{
  "defaultAction": "SCMP_ACT_ERRNO",
  "syscalls": [
    {
      "names": ["read", "write", "open", "close", "stat",
                "mmap", "mprotect", "munmap", "exit_group",
                "futex", "getpid", "clock_gettime"],
      "action": "SCMP_ACT_ALLOW"
    }
  ]
}
```

```
# Apply custom profile
docker run --security-opt seccomp=myprofile.json myapp

# Default Docker seccomp blocks ~44 dangerous syscalls
# including: ptrace, kexec_load, create_module, init_module
```

> The default Docker seccomp profile is a good starting point. Build a custom profile for production using `strace` to capture only the syscalls your app actually needs.

Security Checklist — Day 1

Before shipping any container image, verify:

- [ ] Base image scanned — no CRITICAL/HIGH CVEs
- [ ] Image signed with cosign
- [ ] Dockerfile runs as non-root (`USER`)
- [ ] No secrets baked into layers
- [ ] `--read-only` filesystem enabled
- [ ] `--cap-drop ALL` applied
- [ ] `no-new-privileges` set
- [ ] Resource limits set (memory, CPU, PIDs)
- [ ] Image tagged with specific version (not `:latest`)
- [ ] Registry access authenticated and scoped

Day 1 — Summary

Topic	Key Takeaway
Containers vs VMs	Shared kernel, namespace isolation, near-zero overhead
Kernel namespaces	PID, NET, MNT, UTS, IPC, USER — the six isolation primitives
cgroups	CPU, memory, PID limits enforced at kernel level
Image layers	Union filesystem — layers shared, CoW on write
Dockerfile	Multi-stage builds, pin tags, clean apt caches
Registries	Never use `:latest` in production; prefer digest pinning
TLS	1.3 only; mTLS for east-west traffic
Vulnerability scanning	Trivy in every CI pipeline, fail on CRITICAL/HIGH

Day 1 — Exercises

- **Exercise 1** — Build, tag, scan, and push a container image
- **Exercise 2** — Harden a running container with capabilities and seccomp

See `exercises/day-1/` for full lab instructions.